题目 0：
Lecture 07 · Files 文件

> Advanced Programming — 张子邦（暨南大学光电工程系）
> 文件的概念、文件夹与文件系统、文本文件与二进制文件、字符编码、Python 文件读写（open/close、基本模式、文件指针 tell/seek）

一、什么是文件（What is a File）

在计算机中，****文件（file）**** 是存储在存储设备（硬盘、SSD、U 盘）上、被计算机当作****单一单元****处理的****命名的数据集合****。

****文件的关键特征（Key characteristics）**：**

特征	说明
名称与扩展名（Name & extension）	如 `report.docx`，扩展名 `.docx` 通常暗示格式或打开它的程序
内容（Content）	实际数据：文本、图像、视频、代码或原始字节
元数据（Metadata）	关于文件的信息：大小、创建/修改日期、权限等
位置（Location）	文件系统中的路径，如 `C:\Users\Alex\Documents\report.docx`
类型（Type）	大体分为****文本文件****（人类可读字符）和****二进制文件****（供程序解读的编码数据）

二、文件与文件夹（Files vs. Folders）

****文件夹（folder / directory）**** 是容纳文件和其他文件夹的容器。文件系统把所有内容组织成****层次结构（hierarchy）****，由操作系统统一管理文件的创建、读取、写入和删除。

****空文件/空文件夹占存储空间吗？****——占。空文件和空文件夹都会消耗少量存储空间，只是小到操作系统可能显示为 "0 字节"。具体行为取决于文件系统（如 Windows 的 NTFS、Linux 的 ext4）。

三、什么是文件系统（What is a File System）

****文件系统（file system）**** 是操作系统用来在存储设备上****组织、存储、检索和管理****文件与文件夹的****方法和数据结构****。

可以把它理解为磁盘的"归档系统和规则手册"：它定义数据如何在物理上布局、每个文件存放在哪里、谁能访问它。

> 没有文件系统，硬盘就只是一大串无组织的字节序列——根本无法区分一个文件从哪里开始、到哪里结束。

课件用示意图展示三者关系：**操作系统 ↔ 文件系统 ↔ 文件**，文件系统是 OS 管理文件的中间层；并在 Windows "磁盘属性" 对话框中查看实际使用的文件系统（C 盘为 NTFS）。

常见文件系统：

文件系统	典型用途
NTFS	Windows 系统盘和数据盘
FAT32 / exFAT	U 盘、跨平台可移动介质
ext4	Linux
APFS	macOS 和 iOS
HFS+	旧版 macOS
XFS / Btrfs / ZFS	Linux 服务器、高级功能（快照、校验和）
ISO 9660 / UDF	光盘（CD/DVD）

文件系统的六大核心职责：

1. **命名与路径（Naming and paths）**：给文件命名并组织成层次结构（如 `C:\Users\Alex\Documents\report.docx`）
2. **空间管理（Space management）**：跟踪磁盘上哪些块/簇（blocks/clusters）是空闲、已用或保留的
3. **元数据存储（Metadata storage）**：记录文件大小、时间戳、权限、所有权
4. **访问控制（Access control）**：按用户/组强制执行读/写/执行权限
5. **可靠性（Reliability）**：处理碎片、日志（journaling，崩溃恢复）和错误检查
6. **抽象（Abstraction）**：隐藏底层硬件，程序只需说"打开这个文件"，不必面对磁盘扇区

文件元数据存在哪里？（以 NTFS 为例）：元数据集中存储在 **MFT（Master File Table，主文件表）** 中。每个文件或目录对应 MFT 中的一条记录，包含文件名、时间戳、权限等属性。可用命令 `fsutil fsinfo ntfsinfo C:` 查看 NTFS 信息。

四、Python 中的文件类型：文本 vs 二进制

	文本文件（Text File）	二进制文件（Binary File）
典型扩展名	`.txt`	`.bin`
内容	人类可读的字符	原始字节（如 `01010101…`）
编码	按标准字符编码方案存储	由特定程序或特定格式来解读

文本文件（Text File）：最简单、最基础的文件类型之一，把数据存储为**人类可读字符的序列**，字符按标准字符编码方案编码。

二进制文件（Binary File）：内容以**原始字节**存储、需要**由特定程序或按特定格

式**来解读的文件——而不是当作人类可读的文本来对待。

课件演示了同一个 `.pptx` 文件在两种工具下的样子：
- 用**文本编辑器（Notepad++）**打开：满屏乱码控制字符——因为它不是文本
- 用**十六进制编辑器（WinHex）**打开：可见字节级布局，文件头 `PK`（zip 魔数）和 `[Content_Types].xml` 等结构清晰可见

五、字符编码（Character Encoding）

文本文件归根结底只是**字节**；**编码（encoding）** 告诉软件如何把字节映射为字符。编码不匹配是"乱码（mojibake）"的常见原因——比如 `文件` 显示成 `æ–‡ä»¶`（UTF-8 字节被当成 Latin-1 解读）。

编码	描述
ASCII	7 位，128 个字符（英文字母、数字、基本标点）
UTF-8	变长，向后兼容 ASCII，支持几乎所有语言和 emoji，**当今的主流标准**
UTF-16 / UTF-32	定长/变长，用于部分 Windows 和遗留场景
Latin-1 (ISO-8859-1)	8 位西欧字符
GBK / GB2312	中文字符编码（遗留；如今首选 UTF-8）

UTF-8 字节数计算示例："这是 1 个文本文件" 含 7 个汉字 + 1 个数字。UTF-8 中，常用 CJK 字符（U+4E00–U+9FFF 范围）每个编码为 3 字节，拉丁字符每个 1 字节：

```
7 × 3 + 1 × 1 = 22 字节
```

六、在 Python 中读写文件（open / close）

可以用 `open()` 和 `close()` 手动管理文件。**关键规则：一定要调用 `close()`**——否则数据可能没有刷写（flush）到磁盘，且文件句柄会泄漏。最安全的模式是 **try/finally**，即使中途出错也能保证文件被关闭：

```python
f = open('example.txt', 'r', encoding='utf-8')
try:
    content = f.read()
    print(content)
finally:
    f.close()
```

七、open() 函数

内置函数 `open()` 是 Python 文件操作的主要入口：

```python
open(filepath, mode='r', buffering=-1, encoding=None, \
     errors=None, newline=None, closefd=True, opener=None)
```

核心参数：

- **filepath（必填）**：要打开的文件路径，可以是字符串或 path-like 对象，是唯一的必选参数
- **mode（默认 `'r'`）**：字符串，决定文件的打开方式

文本模式打开（注意 `r'...'` 原始字符串避免 `\` 转义问题）：

```python
filepath = r'c:/myfile.txt'
file = open(filepath, mode='r', encoding='utf-8')
```

二进制模式打开（mode 加 `b`，且不指定 encoding）：

```python
filepath = r'c:/myfile.bin'
file = open(filepath, mode='rb')
```

八、四种基本模式（The Fundamental Modes)

基本模式决定了对文件的主要操作：

模式	名称	行为	文件指针初始位置
`'r'`	Read（读，默认）	只读。**文件必须存在**，否则抛出 `FileNotFoundError`	开头
`'w'`	Write（写）	只写。文件存在则**先截断（清空）**；不存在则新建	开头
`'a'`	Append（追加）	只写。文件存在则指针在**末尾**，新数据追加、不覆盖已有内容；不存在则新建	末尾
`'x'`	Exclusive Creation（排他创建）	类似 `'w'`，但文件已存在时**失败**并抛出 `FileExistsError`，用于防止意外覆盖	开头

九、文件指针（The File Pointer）

文件打开后有一个**文件指针**标记下一次读写的位置：`'r'`/`'w'` 模式指针从**文件开头**开始，`'a'` 模式指针从**文件末尾**开始。

tell()：我现在在哪？

`tell()` 返回文件指针的当前位置，其含义**完全取决于打开文件的模式**：

- **二进制模式**（`'rb'`、`'wb'` 等）：返回整数，表示距文件开头的**精确字节偏移量**——直观、可直接解读
- **文本模式**（`'r'`、`'w'` 等）：返回**不透明数字（opaque number）**——不能当作字节数或字符数解读，唯一合法用途是**传回给 `seek()`** 以回到该位置

seek()：跳转到指定位置

`seek(offset, whence)` 移动指针，`whence`（默认 0）指定参考点：

whence	常量	参考点	offset 约束
0	`os.SEEK_SET`	文件**开头**	必须 ≥ 0
1	`os.SEEK_CUR`	**当前位置**	可正可负
2	`os.SEEK_END`	文件**末尾**	通常为负

> 注：课件第 25、26 页内容相同（seek 重复页），以上为合并整理。

十、要点回顾

1. 文件 = 命名的数据集合；五要素：名称与扩展名、内容、元数据、位置、类型
2. 文件系统是 OS 组织和管理文件的"规则手册"——命名路径、空间管理、元数据、访问控制、可靠性、抽象；NTFS 的元数据集中在 MFT
3. 文本文件是人类可读字符的编码序列（UTF-8 主流，CJK 每字 3 字节）；二进制文件是原始字节，需特定程序/格式解读（用十六进制编辑器查看）
4. `open()` 打开文件，`try/finally` 保证 `close()`；`'r'`/`'w'`/`'a'`/`'x'` 四种基本模式各有指针行为和存在性要求
5. `tell()` 报告指针位置（二进制模式 = 字节偏移，文本模式 = 不透明数字），`seek(offset, whence)` 移动指针

题目 1：成员函数 loadFromSavedFile(filepath)
代码：

```
    def loadFromSavedFile(self, filepath, char_index=None):
        """从仙剑 98 存档文件中按偏移读取指定角色的属性值。

        偏移公式：绝对偏移 = PAGE2(0x200) + 字段基址 + 2 × 角色序号
        所有数值为 16 位小端整数。
```

```
        :param filepath:     .RPG 存档文件路径
        :param char_index:  角色序号  0–4（0=李逍遥  … 4=阿奴）；若不传则用 self.char_index，
                            两者都未设时默认 0（李逍遥）
        :raises IndexError:             char_index 超出 0–4 范围
        :raises FileNotFoundError:      filepath 不存在
        """
        if char_index is None:
            char_index = self.char_index
        if char_index < 0 or char_index >= len(CHAR_NAMES):
            raise IndexError(f'char_index 须在 0–{len(CHAR_NAMES)-1} 范围内')
        self.char_index = char_index
        self.name       = CHAR_NAMES[char_index]

        with open(filepath, 'rb') as f:
            buf = f.read()

        for attr, base in FIELD_OFFS.items():
            off = PAGE2 + base + 2 * char_index
            val = struct.unpack_from('<H', buf, off)[0]
            setattr(self, attr, val)

        # 数据一致性兜底
        self.is_alive = self.hp > 0
        self.is_defending = False
```

运行结果：

```
PS E:\Advanced programming> cd "e:\Advanced programming\作业"
>> python Player.py
读取存档：..\Pal\Pal\pal\0.RPG

[李逍遥] Lv1 HP 150/150 MP 100/100 五维 武33 灵20 防32 速28 运32  → char_0_李逍遥.txt
遥.txt
[赵灵儿] Lv5 HP 28/240 MP 10/240 五维 武20 灵64 防28 速55 运18  → char_1_赵灵儿.txt
[林月如] Lv3 HP 220/220 MP 80/80 五维 武38 灵22 防35 速36 运28  → char_2_林月如.txt
[巫后] Lv48 HP 810/810 MP 790/790 五维 武168 灵256 防136 速132 运60  → char_3_巫后.txt
[阿奴] Lv28 HP 480/480 MP 410/410 五维 武168 灵220 防110 速160 运150  → char_4_阿奴.txt
PS E:\Advanced programming\作业>
```

题目 2：export2Dict(filepath)

代码：

```
    def export2Dict(self, filepath):
        """将角色属性以 JSON 风格文本写入 filepath，格式示例：
```

```
        {
          "name": "李逍遥",
          "HP": 100,
          "HP_max": 150,
          ...
          "luck": 20
        }

    注意：文件内是纯文本字典风格，不是严格的 JSON 文件。
    """
    lines = ['{']
    lines.append(f'  "name": "{self.name}",')
    attrs = list(FIELD_OFFS.keys())              # 保持存档字段顺序
    for i, attr in enumerate(attrs):
        key  = JSON_KEYS.get(attr, attr)
        val  = getattr(self, attr)
        tail = ',' if i < len(attrs) - 1 else ''
        lines.append(f'  "{key}": {val}{tail}')
    lines.append('}')
    with open(filepath, 'w', encoding='utf-8') as f:
        f.write('\n'.join(lines))

# —— 命令行快速验证（python Player.py）——
if __name__ == '__main__':
    import os, glob
    saves = glob.glob(r'..\Pal\Pal\pal\*.RPG') + glob.glob(r'..\Pal\Pal\pal\*.rpg')
    if not saves:
        print('未找到 .RPG 存档，请先放置 Pal\\Pal\\pal\\*.RPG')
    else:
        save = saves[0]
        print(f'读取存档：{save}\n')
        for idx in range(len(CHAR_NAMES)):
            p = Player()
            try:
                p.loadFromSavedFile(save, idx)
            except IndexError:
                break
            out = f'char_{idx}_{CHAR_NAMES[idx]}.txt'
            p.export2Dict(out)
            print(f'[{CHAR_NAMES[idx]}] Lv{p.level} HP {p.hp}/{p.hp_max} '
                  f'MP {p.mp}/{p.mp_max} 五维 '
                  f'武{p.offense} 灵{p.spirit} 防{p.defense} 速{p.speed} 运{p.luck}'
                  f'  → {out}')
```

运行结果：


```json
{
  "name": "李逍遥",
  "level": 1,
  "HP_max": 150,
  "MP_max": 100,
  "HP": 150,
  "MP": 100,
  "offense": 33,
  "spirit": 20,
  "defense": 32,
  "speed": 28,
  "luck": 32
}
```

```json
{
  "name": "巫后",
  "level": 48,
  "HP_max": 810,
  "MP_max": 790,
  "HP": 810,
  "MP": 790,
  "offense": 168,
  "spirit": 256,
  "defense": 136,
  "speed": 132,
  "luck": 60
}
```